SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION WITH OPENCV

COURSE CODE: DSA0204

COURSE OUTCOME COVERED

CO5: *Understand video processing - motion computation - 3D vision - geometry. (BTL 6)*

Assessment Tool Weightage: 40%

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

Code of Conduct:

I K Sampath Reddy (192424285) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____K Sampath Reddy_____

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Real-Time Face Recognition System (Edge Deployment)

Constraints:

- Process live video streams in real time (<30 FPS latency)
- Work under varying lighting conditions
- Use limited memory and CPU resources
- Detect and recognize faces from a predefined dataset

Proposed OpenCV Pipeline:

1. **Image Acquisition:** Capture frames from a USB/CSI camera using cv2.VideoCapture with a threaded reader (imutils.VideoStream) so I/O does not block processing; frame resolution is downscaled to 320x240 to cut per-frame cost.
2. **Preprocessing:** Convert each frame to grayscale (cv2.cvtColor) and apply CLAHE (cv2.createCLAHE) instead of plain histogram equalization, since CLAHE normalizes local contrast without over-amplifying noise under uneven lighting.
3. **Feature Detection:** Use OpenCV's DNN face detector (quantized res10 SSD Caffe model) instead of a deep embedding network; run full detection every 5th frame and track the face region in between using a lightweight KCF tracker to save CPU cycles.
4. **Recognition:** Match detected faces against a pre-enrolled dataset using the LBPH Face Recognizer (cv2.face.LBPHFaceRecognizer_create), which is far cheaper than deep embeddings and is inherently robust to monotonic lighting change because it encodes local texture patterns rather than raw pixel intensity.
5. **Output:** Draw bounding boxes and identity labels (cv2.rectangle, cv2.putText) on the live stream and log recognized identities with a timestamp.

Justification Against Constraints:

- Real-time (<30 FPS latency): achieved by frame downscaling, detect-then-track scheduling (detection only every 5th frame), and choosing LBPH over deep CNN matching.
- Varying lighting: CLAHE preprocessing plus LBPH's texture-based encoding make recognition tolerant to illumination shifts.
- Limited memory/CPU: quantized SSD detector and LBPH recognizer avoid the RAM and compute footprint of full deep-learning pipelines, making the system Raspberry-Pi feasible.
- Predefined dataset recognition: enrollment images are pre-processed once offline into LBPH histograms, so runtime cost is just a nearest-histogram comparison.

2. Automated Video Surveillance with Alert System

Constraints:

- Detects motion and unusual activities
- Generates alerts in real time
- Works with standard CCTV video feeds
- Minimizes false positives in dynamic environments

Proposed OpenCV Pipeline:

1. **Image Acquisition:** Read the RTSP/CCTV stream with cv2.VideoCapture; buffer only the latest frame to avoid processing stale, queued frames.
2. **Preprocessing:** Apply Gaussian blur (cv2.GaussianBlur) to suppress sensor noise, then feed frames into an adaptive background model (cv2.createBackgroundSubtractorMOG2, with shadow detection enabled).

3. **Motion/Feature Detection:** Extract the foreground mask, clean it with morphological opening and closing (cv2.morphologyEx) to remove speckle noise, then find contours (cv2.findContours) and discard blobs below an area threshold.
4. **Analysis:** Track surviving blobs across frames (centroid tracking) and require motion to persist for a minimum number of consecutive frames before it is classified as an 'unusual activity' rather than transient noise.
5. **Alerting:** On a qualifying event, capture a snapshot, push a real-time notification (webhook/email), and log the event with a timestamp.

Justification Against Constraints:

- Motion/unusual activity detection: MOG2 background subtraction plus contour analysis reliably isolates moving foreground objects.
- Real-time alerts: all stages use lightweight OpenCV primitives, so the pipeline keeps pace with standard CCTV frame rates and can fire notifications immediately upon a qualifying event.
- Standard CCTV feeds: cv2.VideoCapture natively supports RTSP/ONVIF streams, so no custom decoder is required.
- Minimizing false positives in dynamic backgrounds: MOG2's adaptive learning rate absorbs gradual scene changes (e.g., swaying trees, lighting drift), while the area-threshold and multi-frame persistence rules filter out short-lived noise such as flickering lights.

3. OCR System for Low-Quality Documents

Constraints:

- Extracts text from low-resolution, noisy scanned images
- Handles varying fonts and illumination
- Works efficiently for batch processing

Proposed OpenCV Pipeline:

1. **Image Acquisition:** Loop over the input directory and load each scanned page with cv2.imread for batch processing.
2. **Preprocessing:** Convert to grayscale, denoise with cv2.fastNlMeansDenoising (tuned for scanned-document noise), then correct any rotation with a Hough-transform-based deskew step.
3. **Enhancement:** Normalize illumination with CLAHE, then binarize using cv2.adaptiveThreshold (Gaussian, rather than a single global threshold) so the system copes with uneven scan lighting.
4. **Segmentation:** Use dilation (cv2.dilate) to merge nearby characters into words/lines, then contour detection to isolate text regions/lines for the recognizer.
5. **Recognition:** Pass each cleaned region to a text recognition engine (e.g., pytesseract) and reassemble the output in reading order; process the whole folder in a batch loop, saving each result to a text file.

Justification Against Constraints:

- Low-resolution, noisy images: non-local means denoising plus adaptive thresholding recovers clean strokes without erasing thin characters.
- Varying fonts/illumination: CLAHE plus adaptive (local) thresholding responds to local brightness rather than a single global cutoff, so both bright and dark regions of the same page are handled correctly, regardless of font weight.
- Batch efficiency: the pipeline is stateless per image and expressed as simple OpenCV calls, so it parallelizes trivially across a directory of scans (e.g., with multiprocessing).

4. Facial Expression Recognition for Mobile Application

Constraints:

- Limited processing power (mobile CPU)
- Real-time performance
- Robustness to slight head movement and lighting changes

Proposed OpenCV Pipeline:

1. **Image Acquisition:** Capture frames from the front camera at a reduced resolution suitable for mobile inference.
2. **Preprocessing:** Detect the face with a lightweight DNN detector, then align it using eye-landmark-based rotation correction so expression features are compared in a normalized pose.
3. **Feature Extraction:** Instead of a heavy CNN, extract Histogram of Oriented Gradients features (cv2.HOGDescriptor) or use a quantized MobileNet-style micro-CNN (TFLite/OpenCV DNN, INT8) sized for mobile CPUs.
4. **Classification:** Feed features into a lightweight classifier (SVM or the micro-CNN's final layer) to predict happy/sad/neutral.
5. **Temporal Smoothing:** Apply majority voting over the last N predictions to stabilize the output against transient misclassifications from slight head motion.

Justification Against Constraints:

- Limited mobile CPU: HOG+SVM or an INT8-quantized micro-CNN keeps both memory and compute far below a full-size deep network.
- Real-time performance: lightweight feature extraction plus a shallow classifier runs in milliseconds per frame on mobile silicon.
- Robustness to head movement/lighting: face alignment normalizes pose before feature extraction, and temporal smoothing filters single-frame errors caused by momentary lighting flicker or motion blur.

5. Smart Traffic Monitoring System

Constraints:

- Detect vehicles from live video streams
- Track movement across frames
- Count vehicles and display statistics
- Operate continuously with minimal delay

Proposed OpenCV Pipeline:

1. **Image Acquisition:** Read the traffic camera stream via cv2.VideoCapture.
2. **Preprocessing:** Background subtraction (cv2.createBackgroundSubtractorMOG2) followed by morphological cleanup to obtain a clean foreground mask of moving vehicles.
3. **Detection:** Extract contours/bounding boxes from the mask (or, for higher accuracy, run a lightweight vehicle detector) and filter by size to exclude noise.
4. **Tracking:** Assign each detected vehicle a unique ID using centroid tracking (or a built-in tracker such as CSRT) so the same vehicle is not re-detected as new across frames.
5. **Counting & Visualization:** Define a virtual counting line; increment the vehicle counter whenever a tracked centroid crosses it, and draw bounding boxes, IDs, the counting line, and the running statistics overlay with cv2.rectangle, cv2.line, and cv2.putText.

Justification Against Constraints:

- Detection from live streams: MOG2 plus contour filtering reliably isolates vehicles from a mostly static road background.
- Tracking across frames: centroid/ID-based tracking maintains vehicle identity between frames, which is essential for accurate counting.

- Accurate counting & statistics: the line-crossing rule tied to persistent IDs prevents the same vehicle from being counted twice.
- Continuous, low-delay operation: all stages rely on lightweight classical CV operations rather than heavy deep detectors, sustaining real-time throughput on a continuous feed.

6. Real-Time Object Detection on Mobile Device

Constraints:

- Detect objects using the camera
- Run in real time
- Use limited CPU and memory

Proposed OpenCV Pipeline:

1. **Image Acquisition:** Capture camera frames and resize them to the detector's expected input dimensions.
2. **Preprocessing:** Normalize pixel values and convert to the blob format required by cv2.dnn.blobFromImage.
3. **Detection:** Run the frame through OpenCV's DNN module loaded with a mobile-optimized, INT8-quantized model (e.g., MobileNet-SSD or a YOLO-Nano variant).
4. **Postprocessing:** Apply non-maximum suppression (cv2.dnn.NMSBoxes) to remove duplicate/overlapping boxes.
5. **Display:** Overlay bounding boxes and class labels on the live camera feed.

Justification Against Constraints:

- Camera-based detection: cv2.dnn integrates directly with the live capture loop.
- Real-time performance: a mobile-first architecture (MobileNet-SSD/YOLO-Nano) is chosen specifically for its low-latency inference relative to server-scale models.
- Limited CPU/memory: INT8 quantization shrinks both the model's memory footprint and its arithmetic cost; periodic frame skipping can be added if the device is especially constrained.

7. Smart Attendance System using Face Recognition

Constraints:

- Capture faces from a webcam
- Recognize registered users
- Mark attendance automatically

Proposed OpenCV Pipeline:

1. **Image Acquisition:** Capture frames from the webcam with cv2.VideoCapture(0).
2. **Preprocessing:** Detect and align faces (DNN/Haar detector), normalize size and grayscale/CLAHE for illumination.
3. **Recognition:** Compare each detected face against enrolled embeddings/LBPH histograms and compute a similarity/confidence score.
4. **Liveness & Confidence Check:** Require the match confidence to exceed a threshold and be consistent across several consecutive frames (and optionally an eye-blink liveness check) before accepting the identity, which guards against a printed photo being used to spoof the system.
5. **Attendance Logging:** On a confirmed match, write the user ID and timestamp to a CSV/database, and apply a cool-down window so the same person is not logged multiple times in one session.

Justification Against Constraints:

- Webcam capture and recognition of registered users: standard cv2.VideoCapture plus LBPH/embedding matching against the enrolled dataset.
- Automatic marking: successful matches are written directly to the log without manual intervention.
- Avoiding false recognition: the multi-frame confidence consensus and optional liveness check reject low-confidence or spoofed matches, and the cool-down period prevents duplicate log entries for the same person.

8. Motion Detection for Home Security

Constraints:

- Detect motion from a camera feed
- Trigger alerts only for significant movement
- Reduce false alarms

Proposed OpenCV Pipeline:

1. **Image Acquisition:** Continuously read frames from the home camera via cv2.VideoCapture.
2. **Preprocessing:** Gaussian blur to suppress sensor noise, then background subtraction (MOG2) or simple frame differencing to isolate change regions.
3. **Detection:** Morphological opening/closing to clean the mask, followed by contour detection and filtering by a minimum bounding-box area.
4. **Significance Filtering:** Require the qualifying motion to persist for several consecutive frames before it is treated as 'significant', filtering out one-off flicker such as a passing shadow or insect.
5. **Alerting:** Send a push notification with a snapshot only when the significance criteria are met; the background model adapts continuously so gradual lighting shifts (dawn/dusk) are absorbed rather than triggering alerts.

Justification Against Constraints:

- Motion detection: MOG2/frame-differencing plus contour analysis reliably flags pixel-level change.
- Significant-movement-only alerts: the combined area + duration thresholds distinguish real intrusions from trivial motion.
- Reduced false alarms: adaptive background modeling absorbs gradual illumination and outdoor-weather variation, while the persistence rule filters transient noise, together minimizing spurious triggers.

9. Document Scanner Application

Constraints:

- Capture document images
- Enhance readability
- Output clean scanned images

Proposed OpenCV Pipeline:

1. **Image Acquisition:** Capture the document photo from the mobile camera.
2. **Edge/Boundary Detection:** Convert to grayscale, blur (cv2.GaussianBlur), and run Canny edge detection (cv2.Canny) to highlight the document's outline.
3. **Contour Extraction:** Find contours (cv2.findContours), keep the largest, and approximate it to a four-point polygon with cv2.approxPolyDP to locate the page corners.
4. **Perspective Correction:** Compute a transform matrix (cv2.getPerspectiveTransform) from the four corners and warp the image to a flat, top-down view with cv2.warpPerspective.
5. **Enhancement:** Apply adaptive thresholding or CLAHE to produce a clean, high-contrast 'scanned' appearance, then optionally sharpen with an unsharp mask.

Justification Against Constraints:

- Capturing and locating the document: Canny + contour + polygon approximation robustly isolates the page even against varied backgrounds.
- Enhancing readability: perspective warping removes skew/keystoning from an angled photo, and adaptive thresholding/CLAHE compensates for uneven lighting across the page.
- Clean output: the combined pipeline yields a flat, high-contrast image comparable to a genuine flatbed scan.

10. Vehicle Detection in Parking System

Constraints:

- Detect cars entering/exiting
- Track movement
- Update available slots

Proposed OpenCV Pipeline:

1. **Image Acquisition:** Capture video at the entry/exit gate(s) with cv2.VideoCapture.
2. **Preprocessing:** Background subtraction (MOG2) with morphological cleanup to obtain vehicle blobs.
3. **Detection:** Contour/bounding-box extraction, or a lightweight cascade/SSD vehicle detector for higher accuracy in cluttered scenes.
4. **Tracking:** Assign persistent IDs via centroid tracking, augmented with a Kalman filter to predict position through brief occlusion by other vehicles.
5. **Slot Management:** Define a virtual entry/exit line; when a tracked vehicle's trajectory crosses it in the entry direction, decrement available slots, and increment on exit; push the updated count to the display/database.

Justification Against Constraints:

- Detecting entering/exiting cars: background subtraction plus contour/detector isolates vehicles reliably at the gate.
- Tracking movement: ID-persistent tracking with Kalman prediction keeps identity through short occlusions.
- Accurate slot updates with multiple vehicles: direction-aware line-crossing logic tied to unique IDs prevents the same vehicle being counted twice and correctly distinguishes simultaneous entries/exits, minimizing counting errors.

11. Gesture Recognition System

Constraints:

- Capture hand gestures from a webcam
- Identify simple gestures
- Respond in real time

Proposed OpenCV Pipeline:

1. **Image Acquisition:** Capture webcam frames.
2. **Preprocessing:** Isolate the hand using HSV skin-color thresholding (cv2.inRange) or background subtraction against a fixed backdrop.
3. **Feature Extraction:** Find the hand contour, then compute its convex hull and convexity defects (cv2.convexHull, cv2.convexityDefects) to count extended fingers.
4. **Classification:** Map the finger count/defect pattern to a predefined gesture (e.g., 0 defects = fist, 4 defects = open palm), or train a lightweight SVM on contour-shape features for more nuanced gestures.

5. **Output:** Display the recognized gesture label live on the video feed.

Justification Against Constraints:

- Capturing and identifying gestures: HSV segmentation plus convex-hull/defect analysis is a classical, well-established technique for simple hand-gesture recognition.
- Real-time response: convex-hull-based feature extraction is computationally cheap, so the pipeline easily sustains live frame rates without GPU acceleration.

12. Video Frame Quality Enhancement

Constraints:

- Improve video quality in real time
- Reduce noise and enhance contrast

Proposed OpenCV Pipeline:

1. **Image Acquisition:** Read frames from the incoming video stream.
2. **Denoising:** Apply an edge-preserving filter — cv2.bilateralFilter for real-time performance, or cv2.fastNlMeansDenoisingColored when latency budget allows — to suppress noise while keeping edges sharp.
3. **Contrast Enhancement:** Convert to LAB color space, apply CLAHE (cv2.createCLAHE) to the L channel only, then merge and convert back to BGR so contrast improves without distorting color.
4. **Sharpening:** Apply an unsharp-mask kernel (cv2.filter2D) to restore perceived detail lost during denoising.
5. **Output:** Stream/display the enhanced frame.

Justification Against Constraints:

- Real-time improvement: bilateral filtering is chosen over slower non-local-means denoising specifically to fit a real-time budget.
- Noise reduction and contrast enhancement: performing denoising before CLAHE/sharpening (in that order) avoids amplifying noise, since sharpening after denoising enhances real edges rather than residual noise.

13. Multi-Face Tracking System

Constraints:

- Detect multiple faces
- Track them across frames
- Maintain identity

Proposed OpenCV Pipeline:

1. **Image Acquisition:** Capture frames containing multiple people.
2. **Detection:** Run a DNN face detector every k frames to (re-)locate all faces in the scene.
3. **Tracking:** Initialize an independent lightweight tracker (CSRT/KCF via cv2.legacy.MultiTracker) for each detected face and update all trackers every frame in between detections.
4. **Identity Maintenance:** On each detection cycle, match new detections to existing tracks using Intersection-over-Union (IoU) overlap (optionally via the Hungarian algorithm) so the same person keeps the same ID rather than being reassigned a new one.
5. **Output:** Draw each tracked box with its persistent ID.

Justification Against Constraints:

- Detecting multiple faces: periodic DNN detection scales naturally to any number of faces in frame.

- Tracking across frames: per-face lightweight trackers update every frame far more cheaply than re-running detection each frame.
- Maintaining identity: IoU-based association between new detections and existing tracks resolves the correspondence problem, preventing ID switches when faces move, cross paths, or briefly occlude one another.

14. OCR for License Plate Recognition

Constraints:

- Detect license plates
- Extract text
- Work under varying lighting

Proposed OpenCV Pipeline:

1. **Image Acquisition:** Capture the vehicle image/frame from a traffic or gate camera.
2. **Plate Localization:** Apply Canny edge detection and contour analysis, filtering candidate contours by the aspect ratio typical of license plates (or use a Haar-cascade plate detector) to isolate the plate region.
3. **Preprocessing the ROI:** Convert the localized plate to grayscale, apply CLAHE to normalize exposure, and binarize with adaptive thresholding to cope with glare, shadows, or night-time lighting.
4. **Character Segmentation:** Use contour detection on the binarized plate to isolate individual characters.
5. **Recognition:** Pass segmented characters/the plate ROI to an OCR engine (pytesseract) restricted to a plate-appropriate character whitelist (A-Z, 0-9) to improve accuracy.

Justification Against Constraints:

- Detecting plates: aspect-ratio-filtered contour analysis (or a dedicated cascade) efficiently narrows the search to plate-like regions.
- Extracting text: character segmentation plus a constrained-alphabet OCR pass improves recognition accuracy on the small, uniform-font text of a plate.
- Varying lighting: CLAHE and adaptive thresholding specifically target uneven illumination (daylight glare, headlight glare at night), keeping character strokes distinguishable from the background.

15. Real-Time Edge Detection System

Constraints:

- Capture video
- Detect edges in real time
- Display results efficiently

Proposed OpenCV Pipeline:

1. **Image Acquisition:** Capture frames continuously via cv2.VideoCapture, ideally on a separate capture thread so display never blocks on I/O.
2. **Preprocessing:** Apply Gaussian blur (cv2.GaussianBlur) to suppress noise that would otherwise generate spurious edges.
3. **Edge Detection:** Run cv2.Canny with thresholds computed adaptively from the median pixel intensity of each frame (rather than fixed constants), so the result is consistent as scene brightness changes.
4. **Optimization:** Process frames at a modestly reduced resolution and use a producer/consumer threaded pipeline for capture and display to avoid frame drops.
5. **Display:** Render the edge map (or edges overlaid on the original frame) with cv2.imshow in the display loop.

Justification Against Constraints:

- Real-time capture and edge detection: Gaussian blur plus Canny is one of the lightest classical edge-detection pipelines available, easily sustaining live frame rates.
- Efficient display: threaded capture/display and resolution reduction prevent the rendering loop from becoming the bottleneck.
- Optimization/consistency: adaptive (median-based) Canny thresholds keep edge quality stable across varying scene lighting without manual retuning.